Cahier des charges : éditeur et compilateur BASIC évolué pour Amstrad CPC

Version 1.0, projet gratuit et libre, porté par un développeur unique, assisté par IA.

1. Contexte et objectifs

Contexte. L'Amstrad CPC dispose d'un riche patrimoine de jeux, et le BASIC Locomotive reste pour beaucoup la porte d'entrée vers la programmation. Il est pourtant lent, limité pour le graphisme, le son et les sprites, et son éditeur d'origine est rudimentaire. Les débutants qui veulent créer des jeux butent vite sur ces limites et sur une documentation souvent technique et peu illustrée.

Vision. Offrir un environnement de création de jeux complet et accessible : un BASIC évolué, dans l'esprit du Locomotive BASIC, enrichi de nombreuses commandes orientées jeu, avec un éditeur coloré et un compilateur qui produit directement des fichiers exploitables sur matériel réel ou en émulation.

Objectifs.

1. **Éditeur** : éditeur de code avec coloration syntaxique, pensé pour les débutants.
2. **Langage** : BASIC évolué dérivé du Locomotive BASIC, avec des commandes supplémentaires (graphismes, sprites, son, gestion des jeux).
3. **Compilateur binaire** : génération de fichiers .dsk (disquette 3″), de fichiers cassette et de cartouches GX4000.
4. **Bible explicative** : documentation de chaque instruction avec des modèles d'utilisation réels et fonctionnels.
5. **Extensibilité** : architecture capable de s'unir plus tard avec un projet distinct permettant de créer de nouvelles commandes en Z80.

Public cible. Des personnes qui veulent créer des jeux mais ont peu de bases en programmation, et qui progressent grâce à l'outil et à sa documentation. S'y ajoutent les passionnés de rétro-programmation CPC.

Plateforme d'exécution. L'éditeur et le compilateur tournent sur ordinateur moderne (développement croisé). Windows est obligatoire. Linux et macOS sont souhaitables, sans retarder la livraison Windows. Le copier/coller est une exigence de base, y compris depuis et vers d'autres applications.

Critère de réussite global. Un débutant écrit un petit jeu dans l'éditeur sous Windows, le compile en .dsk, cassette ou cartouche GX4000, et le lance sur CPC réel ou émulé en s'aidant uniquement de l'éditeur et de la bible.

2. Périmètre

Inclus.

- Éditeur avec coloration syntaxique, copier/coller, numérotation de lignes ou étiquettes
- BASIC évolué dérivé du Locomotive BASIC, avec commandes ajoutées
- Compilateur binaire générant .dsk, cassette et cartouche GX4000
- Menu de sélection de la machine cible : CPC 464, 664, 6128, 464+, 6128+, GX4000
- Menu de sélection de la ROM et de la disposition clavier : AZERTY, QWERTY et autres variantes nationales existantes (à inventorier)
- Bible de référence avec exemples réels et fonctionnels pour chaque instruction
- Points d'entrée dans l'architecture pour l'union future avec le projet de commandes Z80

Exclus.

- Le projet de commandes Z80 lui-même (seule l'interface d'union est prévue)
- Éditeurs de sprites, de sons ou de niveaux (évolution future possible)
- Un émulateur développé dans le cadre du projet (on s'appuie sur des émulateurs existants)
- Les machines autres que la famille Amstrad CPC

Conséquences du choix de la cible.

Cible	Mémoire	Sorties logiques
CPC 464	64 Ko	Cassette (et .dsk si lecteur externe)
CPC 664	64 Ko	.dsk
CPC 6128	128 Ko	.dsk
CPC 464+	64 Ko	Cassette, .dsk
CPC 6128+	128 Ko	.dsk
GX4000	64 Ko	Cartouche uniquement

Le choix de la cible filtre les formats de sortie proposés, active ou désactive les commandes propres au matériel Plus (palette de 4096 couleurs, sprites matériels, son amélioré), et adapte la mémoire disponible. Les versions exactes du BASIC et du firmware de chaque machine seront vérifiées en phase technique.

3. Parties prenantes et utilisateurs

- **Porteur du projet** : une seule personne, qui assure conception, développement, tests et rédaction de la bible. Décideur unique et valideur de chaque étape.
- **Utilisateurs finaux** : débutants désirant créer des jeux, et passionnés de la communauté CPC.
- **Utilisateur technique** : le futur projet de commandes Z80, qui s'appuiera sur l'architecture d'extension.
- **Communauté** : testeurs et conseillers bénévoles, relecteurs de la bible, retours sur forums.
- **Modèle de diffusion** : gratuit, ouvert, partagé à qui le veut. Une licence libre sera choisie en J0 (par exemple GPL ou MIT pour le code, Creative Commons pour la bible).

Conséquences.

1. Charge réaliste pour une personne : découpage en versions livrables successives.
2. Dépôt public (GitHub ou équivalent) pour faciliter retours et corrections.
3. Vigilance sur les ROM, protégées par des droits : ne pas les redistribuer sans accord, demander à l'utilisateur de fournir les siennes ou recourir à des ROM libres de remplacement.

4. Besoins fonctionnels

Priorités : Indispensable, Important, Souhaitable.

A. Éditeur

Besoin	Priorité
Coloration syntaxique, copier/coller	Indispensable
Numérotation de lignes ou étiquettes	Indispensable
Menus de choix de la machine cible et de la ROM / disposition clavier	Indispensable
Aide intégrée liée à la bible (survol ou touche d'aide sur une instruction)	Important

B. Langage : structure et rapidité

Besoin	Priorité
Boucles plus rapides (bénéfice de la compilation en binaire)	Indispensable
Conditions évoluées : SELECT CASE, et d'autres à lister	Indispensable
Clavier et manette : touche pressée, directions, boutons, multi-touches	Important

C. Graphismes

Besoin	Priorité
CIRCLE, STAR, PAINT et autres formes (ellipse, polygone, rectangle plein…)	Important
Sprites	Important
Collisions	Important
Défilement (scrolling)	Souhaitable
Gestion des 3 modes dans un même écran	Souhaitable, techniquement exigeant

D. Son

Besoin	Priorité
Effets prédéfinis : VROOM, BOOM, SHOT, FALL, WAVE, et d'autres	Important

E. Gestion d'erreurs

Besoin	Priorité
Messages expliquant la cause (variable concernée, valeur fautive, ligne, suggestion), à la compilation et à l'exécution	Important

F. Évolutivité

Besoin	Priorité
Langage conçu pour accueillir de nouvelles commandes sans casser les anciennes, avec numéro de version du langage	Indispensable
Union future avec le projet de commandes Z80	Indispensable (architecture)

G. Mots-clés

Besoin	Priorité
Tous les mots-clés en anglais, anciens comme nouveaux	Indispensable
Liste des nouveaux mots-clés figée dans la bible, en évitant les conflits avec les mots réservés existants	Important

H. Optimisation de taille du compilateur

Besoin	Priorité
Liaison sélective : seules les routines des commandes et fonctions utilisées sont intégrées au binaire	Indispensable
Détection du code et des variables jamais utilisés (avertissement dans l'éditeur)	Souhaitable
Affichage de la taille du binaire et de la mémoire restante selon la cible	Important

Remarques techniques.

- Sprites : gérés par logiciel sur 464, 664 et 6128 ; sprites matériels sur Plus et GX4000. L'usage reste identique pour le programmeur, le compilateur choisit la méthode.
- Les 3 modes dans un écran reposent sur des interruptions synchronisées avec le balayage : délicat, donc classé Souhaitable.
- Bibliothèque modulaire : chaque commande est une routine Z80 indépendante avec la liste de ses dépendances. Ce modèle sert aussi l'union avec le projet Z80 : une nouvelle commande devient un module de plus.

5. Contraintes et exigences non fonctionnelles

Domaine	Exigence
Plateforme	Windows obligatoire ; Linux et macOS si possible
Installation	Simple, idéalement un dossier à décompresser, sans installateur lourd
Performance	Compilation quasi instantanée pour un programme de jeu courant
Compatibilité	Vérifiée sur les émulateurs courants et, si possible, sur matériel réel
Ergonomie	Pensée pour débutants : messages clairs, aide contextuelle, exemples prêts à l'emploi
Langue de l'interface	Menus et messages en français, anglais et espagnol, au choix de l'utilisateur
Langue de la bible	Français uniquement en V1 ; traductions possibles plus tard
Mots-clés du langage	Anglais uniquement
Licence	Libre, à choisir en J0
Dépendances externes	Aucune ou minimes

Conséquences techniques.

1. Tous les textes de l'interface sont stockés dans des fichiers de langue, un par langue : ajouter une langue ne touche pas au programme.
2. Dans la bible, les explications sont séparées des exemples de code, ce qui facilite la traduction ultérieure (réalisée avec l'aide d'une IA, puis relue par la communauté).
3. Les messages d'erreur détaillés contiennent des éléments variables (variables, valeurs) à insérer au bon endroit : soin particulier à prévoir.

6. Budget, délais et jalons

Budget. Aucun budget financier obligatoire. Le principal coût est le temps du porteur : 3 à 4 heures par jour, soit environ 20 à 28 heures par semaine. Frais facultatifs : nom de domaine, achat d'un vrai CPC ou d'un lecteur de cartes pour tests sur matériel réel.

Principe. Pas de date imposée : chaque jalon se termine quand ses critères sont atteints. Les estimations sont des ordres de grandeur, à recalibrer après J1.

Jalon	Contenu	Critère de fin	Estimation
J0	Cahier des charges, choix technologiques, licence, dépôt public	Document validé, prototype « texte → binaire → .dsk → émulateur »	2 à 4 semaines
J1	Éditeur de base (coloration, copier/coller) et compilateur minimal	Un programme de quelques lignes (PRINT, boucles) compile en .dsk et se lance sur émulateur	6 à 10 semaines
J2	Langage de base complet, liaison sélective, cassette, messages d'erreur clairs	Un programme Locomotive BASIC de taille moyenne compile et tourne	8 à 12 semaines
J3	Commandes graphiques et sonores, début de la bible	CIRCLE, STAR, PAINT, effets sonores fonctionnels, exemples documentés	6 à 10 semaines
J4	Sprites, collisions, CPC Plus et GX4000	Un petit jeu avec sprites tourne sur 6128 et en cartouche GX4000	8 à 14 semaines
J5	Défilement, 3 modes dans un écran, bible complète, interface espagnole	Toutes les commandes documentées, interface en 3 langues	8 à 12 semaines

Durée totale indicative : 1 an à 1 an et demi.

Méthode. Un prototype minuscule dès J0 (compiler un seul PRINT et le voir s'afficher dans un émulateur) élimine le plus gros risque technique. Une première version publique dès J1 attire des testeurs et apporte des retours.

7. Livrables et critères d'acceptation

Livrable	Critère d'acceptation
Éditeur + compilateur (Windows, puis Linux/macOS si possible)	Se lance sans installation complexe, compile et produit les fichiers de sortie
Fichiers de sortie	.dsk, cassette et cartouche GX4000 valides, vérifiés sur émulateur
Bibliothèque de commandes (modules Z80)	Chaque module déclare ses dépendances et n'est lié que s'il est utilisé

Livrable	Critère d'acceptation
Bible (en français)	Écrite une seule fois en Markdown, puis générée en pages HTML, en PDF et intégrée à l'éditeur pour l'aide contextuelle. Chaque instruction a une description, une syntaxe et au moins un exemple fonctionnel testé
Exemples de jeux	Au moins 3 petits jeux complets, de difficulté croissante
Interface en 3 langues	Français, anglais, espagnol pour menus et messages
Dépôt public	Code source, licence, aide à l'installation, suivi des problèmes

Les exemples de la bible sont du vrai code : le compilateur les compile à chaque modification de la bible, ce qui sert de test automatique.

Critère d'acceptation de la V1.0. Un débutant installe l'outil sous Windows, ouvre un des 3 jeux d'exemple, le modifie en s'aidant de la bible intégrée, le compile pour la cible de son choix et le lance sur émulateur.

8. Risques et hypothèses

Risque	Gravité	Parade
Projet mené seul : épuisement ou dispersion	Élevée	Jalons courts, publication précoce, communauté
Ampleur du compilateur (traduction BASIC vers Z80 fiable)	Élevée	Prototype en J0, périmètre de langage limité en J1
Choix du langage de l'outil, alors que le porteur connaît BASIC et Z80	Moyenne	Développement assisté par IA ; architecture en couches ; essai de C# en J0 (comparé à Python si l'essai déçoit) ; documents d'architecture à jour ; tests automatiques ; Git
Dépendance à l'IA pour comprendre le code produit	Moyenne	Code simple et commenté, structure documentée, explications sur demande
Droits sur les ROM Amstrad	Moyenne	Ne pas les redistribuer, ou utiliser des ROM libres

Risque	Gravité	Parade
Compatibilité Locomotive BASIC (flottants, chaînes, mémoire)	Moyenne	Définir ce qui est compatible et documenter les écarts
Format cartouche GX4000 et particularités du matériel Plus	Moyenne	Étudier tôt la documentation, tester sur émulateur
Les 3 modes dans un écran (interruptions)	Moyenne	Classé en J5, abandonnable sans bloquer la V1
Traductions imparfaites	Faible	Relecture par la communauté

Hypothèses.

- Le porteur dispose de 3 à 4 heures par jour sur la durée.
- Des émulateurs gratuits et fiables du CPC et du GX4000 restent disponibles.
- Les ROM nécessaires sont obtenables légalement par les utilisateurs.

Choix technologique provisoire. C# avec .NET et Avalonia, organisé en trois couches : un cœur de compilateur sans interface (analyse du BASIC, génération Z80, liaison sélective, écriture des formats), un outil en ligne de commande pour les tests automatiques et l'union avec le projet Z80, et l'éditeur graphique posé au-dessus. Décision confirmée ou révisée à l'issue de l'essai de J0 (fenêtre avec éditeur coloré, et compilation d'un seul PRINT en .dsk). Deux sous-questions à trancher en J0 : assembleur Z80 interne ou outil libre existant, et écriture directe ou non des formats .dsk, cassette et cartouche.

9. Organisation et gouvernance

Domaine	Décision
Décideur	Le porteur du projet, seul
Rôle de l'IA	Assistance à la conception, à l'écriture du code, aux tests, à la documentation et aux traductions ; le porteur reste chef de projet et valide chaque étape
Outils de suivi	Dépôt public avec liste de tâches par jalon (GitHub Projects ou équivalent)
Rythme de travail	Sessions de 3 à 4 h, chacune avec un objectif précis et une courte note de fin (fait, reste à faire)

Domaine	Décision
Documents de référence	Cahier des charges, document d'architecture, journal des décisions, bible. Ils servent de mémoire du projet et sont fournis à l'IA au début de chaque session
Retours de la communauté	Testeurs et conseillers choisis par le porteur, qui relaient leurs retours sur certaines parties du code ; forums CPC et suivi des problèmes sur le dépôt
Contributions de code	Pas de contributions directes au début : le code reste sous la maîtrise du porteur. Les avis, tests et relectures sont intégrés par lui, avec l'aide de l'IA. Ouverture possible plus tard, avec règles écrites dans le dépôt
Versions	Numérotation claire (0.1, 0.2… jusqu'à 1.0) et fichier d'historique des changements
Sauvegardes	Dépôt en ligne et copie locale régulière

Prochaines étapes (J0)

1. Créer le dépôt Git public et choisir la licence.
2. Installer les outils (.NET, un éditeur de code, un émulateur CPC et GX4000).
3. Réaliser le prototype minimal : un PRINT compilé, écrit dans un .dsk, lancé sur émulateur.
4. Rédiger le document d'architecture et le journal des décisions.
5. Confirmer ou réviser le choix du langage de développement.